



INSTITUTO POLITÉCNICO NACIONAL



Procesamiento de Señales

Práctica 2.

**Discretización de un Filtro Pasa-Bajas
mediante los Métodos de Euler y Tustin**

INTEGRANTES:

Gutiérrez Sánchez Angélica

Hernández Jiménez Erick Yael

GRUPO 5BV1

Miguel Ángel Castillo Martínez

20 de mayo de 2025

I. RESUMEN

Este documento explora los fundamentos teóricos y prácticos de la discretización de un filtro pasa-bajas utilizando los métodos de Euler y Tustin. Se detallan las ecuaciones matemáticas que permiten convertir un sistema continuo en uno discreto, facilitando su implementación en entornos digitales. Se realizaron simulaciones en MATLAB de un filtro pasa-bajas activo en tiempo discreto, utilizando estos métodos de discretización. Estas simulaciones permitieron analizar y comparar el comportamiento del filtro bajo diferentes condiciones, proporcionando una comprensión más profunda de los efectos de cada método de discretización en la respuesta del sistema.

Palabras clave: *Filtro pasa-bajas, discretización, Euler, Tustin.*

II. MARCO TEÓRICO

II-A. Filtros

Un filtro es un dispositivo diseñado para seleccionar componentes de frecuencia dentro de una señal, permitiendo el paso (o en algunos casos la amplificación) de aquellas que se encuentran dentro de un rango determinado, y atenuando o bloqueando las que se encuentran fuera de ese rango [1].

II-A1. Clasificación: Es importante mencionar que, los filtros pueden clasificarse de distintas maneras, dependiendo del criterio de análisis. Una de las formas más comunes es según el rango de frecuencias que permiten pasar, lo que da lugar a filtros pasa-bajas, pasa-altas, pasa-banda y rechaza-banda, cada uno con aplicaciones específicas en el procesamiento de señales y sistemas de control [2].

Otra clasificación relevante es por el tipo de componentes utilizados: los filtros pasivos están conformados únicamente por resistencias, inductores y capacitores, mientras que los activos utilizan amplificadores operacionales junto con resistencias y capacitores, lo que permite obtener ganancia y mayor control sobre la respuesta del sistema [3].

Los filtros también pueden categorizarse por su orden, el cual determina la pendiente de atenuación fuera de la banda pasante y la complejidad del diseño [4]. Además, existen clasificaciones según el tipo de función matemática o respuesta: filtros Butterworth (respuesta plana), Chebyshev (rizado en banda), Elípticos (rizado en ambas bandas), y Bessel (fase lineal) [5].

Por otro lado, se distingue entre filtros analógicos, que operan en el dominio continuo, y digitales, que operan en el dominio discreto y pueden implementarse por software [6].

Finalmente se encuentra la clasificación según el orden del filtro, el cual se refiere al grado del polinomio en el denominador de su función de transferencia y está directamente relacionado con la cantidad de elementos reactivos (como capacitores o inductores) o retardos en el sistema; a mayor orden, mayor es la pendiente de atenuación fuera de la banda pasante [3].

II-B. Filtro pasa-bajas

Este tipo de filtro tiene la propiedad de transmitir aquellas componentes de señales de baja frecuencia, incluyendo las señales de corriente directa, mientras que las componentes de altas frecuencias incluyendo las infinitas son bloqueadas [1].

II-C. Función de transferencia

La función de transferencia es una expresión matemática que caracteriza la relación entre la entrada y la salida de sistemas lineales invariantes en el tiempo. Se define como el cociente entre la transformada de Laplace de la salida y la transformada de Laplace de la entrada, bajo la suposición de condiciones iniciales cero [7].

Esta herramienta permite analizar, modelar e implementar sistemas físicos, electrónicos o de control de manera matemática, ya que describe su comportamiento en el dominio de la frecuencia en lugar del tiempo.

Matemáticamente se expresa como:

$$H(s) = \frac{U_o(s)}{U_i(s)} \quad (1)$$

Donde:

- $H(s)$ es la función de transferencia.
- $U_o(s)$ es la transformada de Laplace de la salida.
- $U_i(s)$ es la transformada de Laplace de la entrada.

II-D. Métodos de discretización

En el diseño de filtros, especialmente en aquellos utilizados para procesamiento de señales, es común partir de una función de transferencia en tiempo continuo. Sin embargo, para su implementación en sistemas digitales como microcontroladores o software, es necesario convertir esta representación continua a una equivalente en tiempo discreto. Este proceso se conoce como discretización.

Existen varios métodos para realizar esta conversión, siendo los más comunes el método de Euler y el método de Tustin. Ambos permiten obtener una función $H(z)$ que conserva, en mayor o menor medida, las características del sistema original.

II-D1. Método de Euler: En este método el proceso de discretización consiste en sustituir la variable s de la función de transferencia continua por su equivalente discreto. Para el caso de Euler hacia atrás, esta sustitución se realiza mediante:

$$s \approx \frac{z-1}{zT} \quad (2)$$

Para el caso de Euler hacia adelante, la sustitución de s corresponde a:

$$s \approx \frac{z-1}{Tz} \quad (3)$$

En donde T corresponde al periodo de muestreo, es decir, el intervalo de tiempo entre dos muestras consecutivas en un sistema digital.

Al aplicar alguna de estas sustituciones de s sobre una función de transferencia $H(s)$, se obtiene una nueva expresión $H(z)$ en el dominio Z . Esta función se simplifica a una forma racional con potencias negativas de z :

$$H(z) = \frac{b_0 + b_1 z^{-1} + \dots + b_M z^{-M}}{1 + a_1 z^{-1} + \dots + a_N z^{-N}} \quad (4)$$

Donde:

- a_i son los coeficientes asociados a la salida del sistema, representan la influencia de las salidas pasadas.
- b_i son los coeficientes asociados a la entrada del sistema, representan el peso de las entradas actuales y pasadas
- N es el orden del denominador o del sistema, indica cuántas salidas pasadas afectan la salida actual $y[n]$.
- M es el orden del numerador, indica cuántas entradas pasadas se consideran para calcular la salida $y[n]$.

Esta forma racional con potencias negativas puede transformarse en una ecuación de diferencias utilizando la equivalencia:

$$f(z)z^{-n} = f[k-n] \quad (5)$$

Esta propiedad establece que multiplicar una función en el dominio Z por una potencia negativa de z equivale

a un desplazamiento temporal hacia el pasado en el dominio discreto.

Aplicando esta propiedad sobre la ecuación (4) se obtiene una ecuación en diferencias que relaciona la salida actual $y[n]$ con las entradas $u[n]$ y las salidas pasadas $y[n-k]$:

$$y[n] = -a_1 y[n-1] - \dots - a_N y[n-N] + b_0 u[n] + b_1 u[n-1] + \dots + b_M u[n-M] \quad (6)$$

Esta ecuación ya es implementable en software para sistemas de procesamiento de señales, ya que permite calcular la salida actual del sistema $y[n]$ a partir de entradas y salidas anteriores, utilizando únicamente operaciones aritméticas básicas.

II-D2. Método de Tustin: Este método se basa en la regla del trapecio para aproximar integrales, lo que proporciona una mejor precisión en la representación del sistema en comparación con otros métodos de discretización [8].

La transformación se hace mediante la sustitución:

$$s \approx \frac{2}{T} \cdot \frac{z-1}{z+1} \quad (7)$$

Al igual que en el método de Euler, la aplicación del método de Tustin sobre una función de transferencia continua conduce a una expresión en el dominio discreto $H(z)$ con la forma de la ecuación 4. Esta representación también puede transformarse fácilmente en una ecuación en diferencias como la ecuación 6, adecuada para su implementación en sistemas digitales.

III. METODOLOGÍA

Para el desarrollo de esta práctica se tomó un enfoque experimental simulando en MATLAB un filtro pasabajas activo en tiempo discreto tomando en cuenta el método de Euler y de Tustin.

Para esta simulación fue necesario contar con algún complemento que interprete matemática simbólica y de simulación, como lo son los complementos **Symbolic Math Toolbox** y **Control System Toolbox** respectivamente.

En la simulación del filtro con ambos métodos de discretización se contempló el siguiente orden:

1. Declaración de constantes.
2. Declaración inicial de la señal de entrada.
3. Declaración inicial de la señal de salida.

4. Pre computación del denominador de la fórmula (por ser constante).
5. Cálculo iterativo del numerador de la fórmula y de la razón.
6. Impresión de la gráfica de comportamiento del filtro pasa bajas

En ambas simulaciones se cuenta con 4 variables de las cuales 3 se manipularon en experimentos distintos para comprender mejor los resultados de la discretización por el método de Euler y el de Tustin.

Los experimentos a realizar son:

- Caso base: $\omega_c = 100$, $k = 1$ y $n_{max} = 25$
- Experimento 1: $\omega_c = 200$, $k = 1$ y $n_{max} = 25$
- Experimento 2: $\omega_c = 50$, $k = 1$ y $n_{max} = 35$
- Experimento 3: $\omega_c = 100$, $k = 2$ y $n_{max} = 25$
- Experimento 4: $\omega_c = 100$, $k = 3$ y $n_{max} = 125$
- Experimento 5: $\omega_c = 100$, $k = 2,5$ y $n_{max} = 50$

Al finalizar los experimentos y obtener las gráficas correspondientes, se compararon los resultados obtenidos.

IV. DISCUSIÓN DE RESULTADOS

Considerando la ecuación original que describe la salida del filtro con respecto a la entrada:

$$\frac{U_o(s)}{U_i(s)} = \frac{k\omega_c^2}{s^2 + (3 - k)\omega_c s + \omega_c^2} \quad (8)$$

Se realizaron las sustituciones y simulaciones correspondientes a cada método.

El resultado de la sustitución correspondiente se encuentra en el Anexo A. Mientras que el código para la simulación con el método de Euler en MATLAB se encuentra en el Anexo B y para la simulación con el método de Tustin se encuentra en el Anexo C.

El resultado de la simulación en el experimento base, considerando las condiciones $\omega_c = 100$, $k = 1$ y $n_{max} = 25$, es:

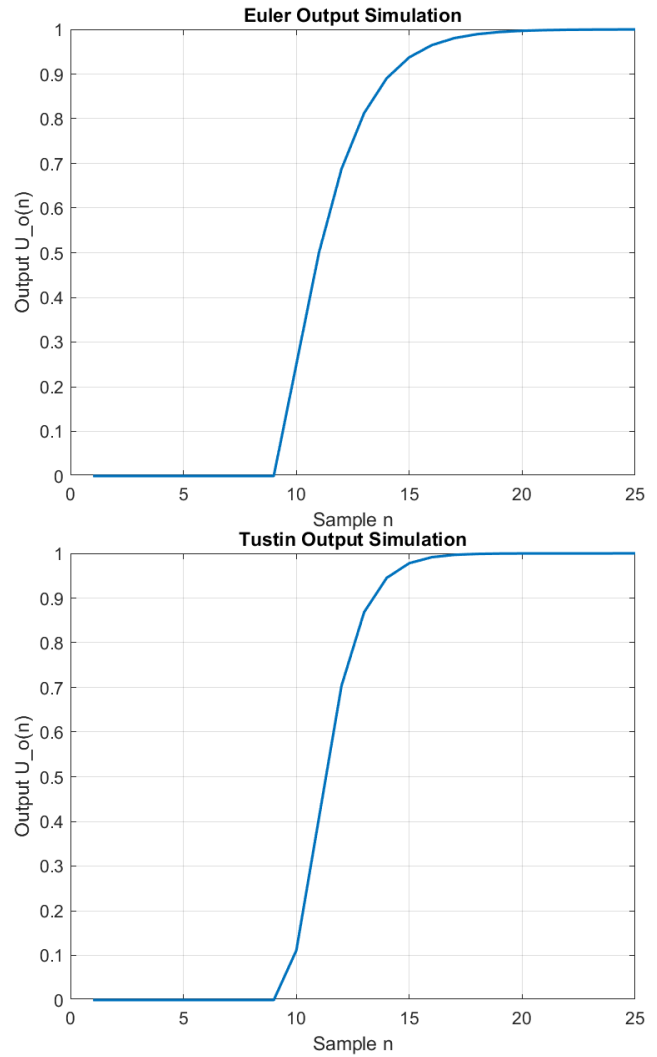


Figura 1: Simulación con caso base.

Inicialmente podemos ver que la función de salida para la discretización con el método de Euler presenta el rasgo visual de mayor suavidad en la curva y una alza más abrupta en contraste con la salida de la discretización de Tustin.

Esta será nuestra referencia para observar cómo los parámetro alteran el comportamiento de la señal de salida.

El resultado de la simulación en el experimento 1, considerando las condiciones $\omega_c = 200$, $k = 1$ y $n_{max} = 25$, es:

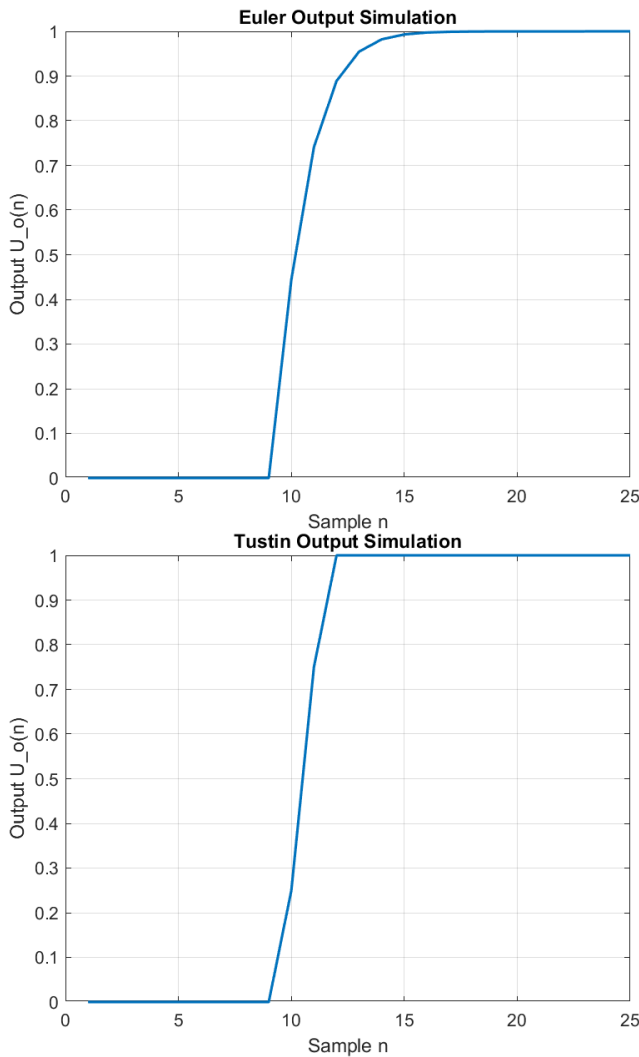


Figura 2: Simulación con experimento 1.

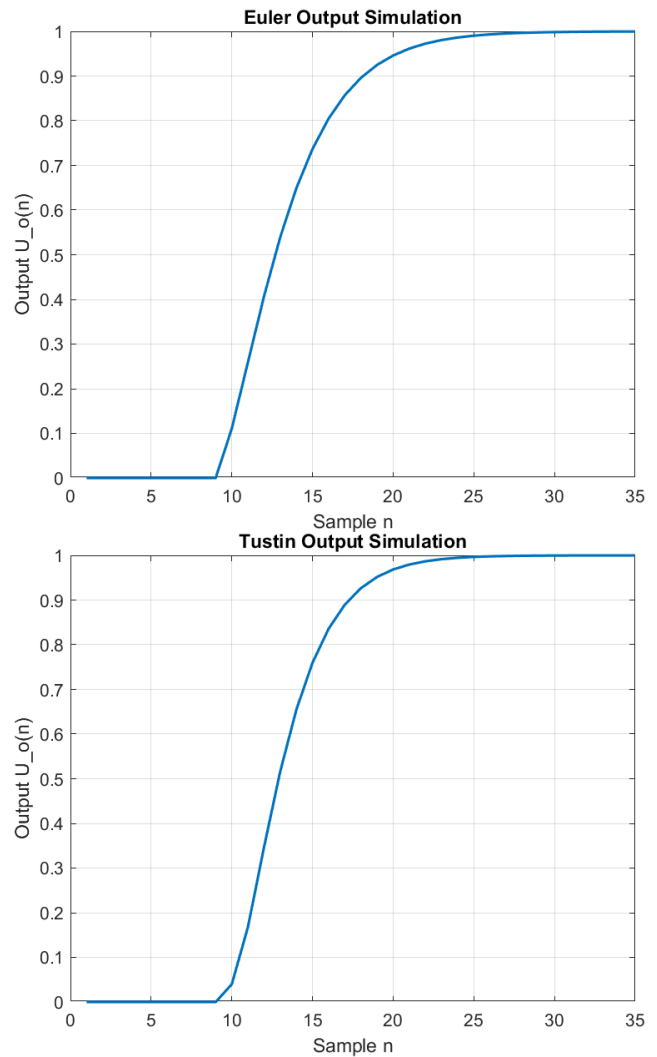


Figura 3: Simulación con experimento 2.

Como se puede observar, las características descritas anteriormente en el caso base se acentuaron, donde la simulación de Euler presenta una curvatura con una pendiente más grande y un cambio asintótico más acelerado.

Es importante mencionar que, en la simulación de Tustin se presenta un comportamiento más cercano al de una función de escalón. Por otro lado, en ambos casos la alza de la curvatura comienza en el mismo punto que en las referencias del caso base, aunque su aproximación asintótica comienza en muestras más tempranas.

El resultado de la simulación en el experimento 2, considerando las condiciones $\omega_c = 50$, $k = 1$ y $n_{max} = 35$, es:

En contraste con el experimento 1, la pendiente en la alza es menos pronunciada incluso que en el caso base para ambos casos, mostrando un comportamiento directamente relacionado con la variable ω_c .

El resultado de la simulación en el experimento 3, considerando las condiciones $\omega_c = 100$, $k = 2$ y $n_{max} = 25$, es:

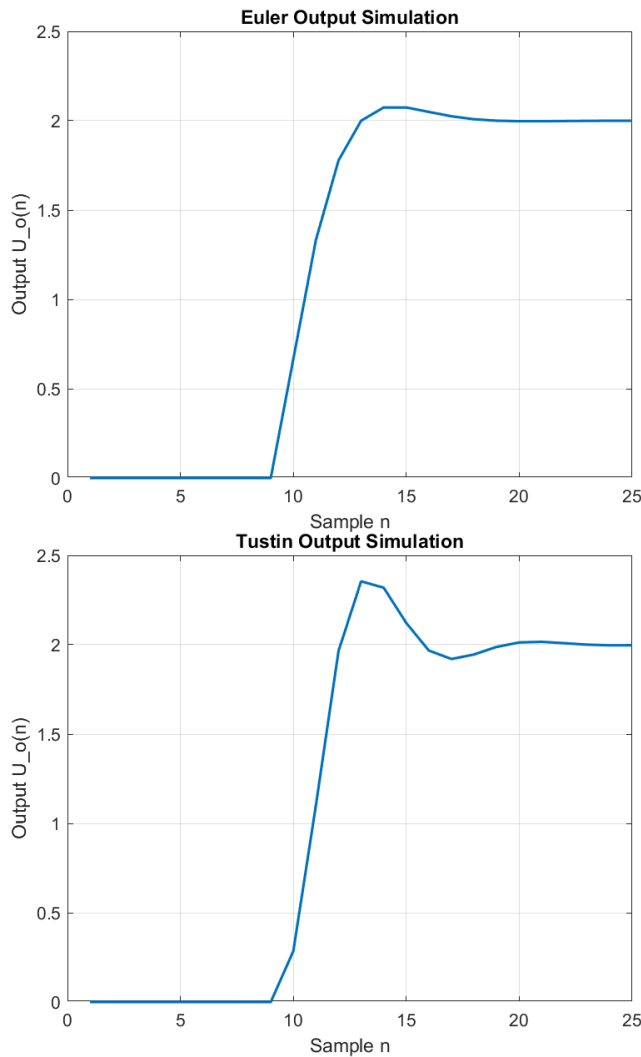


Figura 4: Simulación con experimento 3.

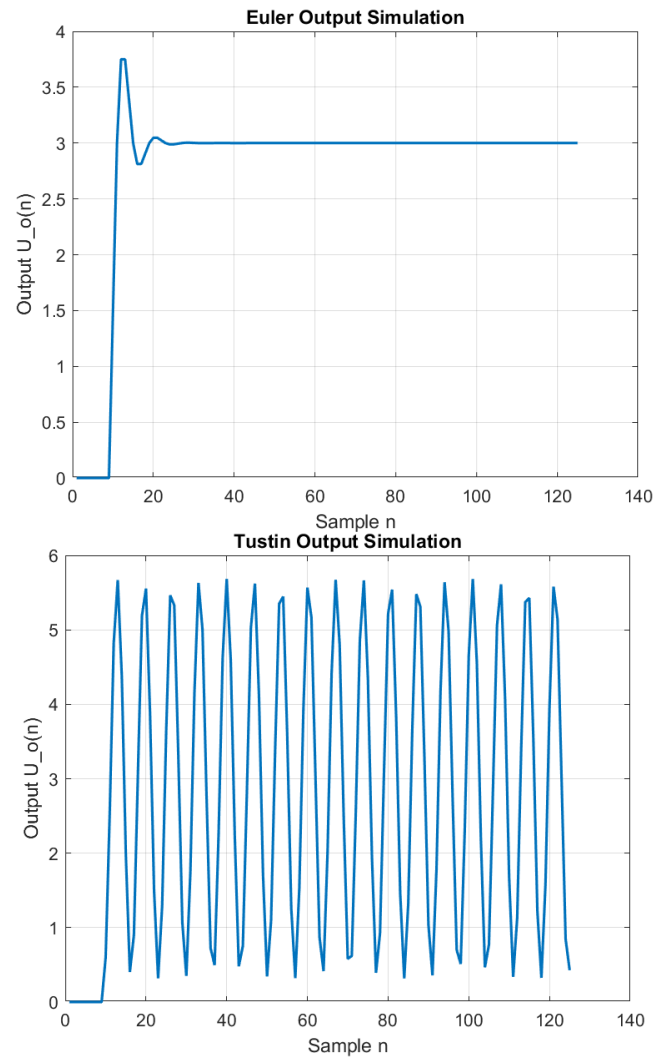


Figura 5: Simulación con experimento 4.

Al cambiar el valor de k se puede notar la generación de una “colina” antes de mantenerse estable la señal al final del muestreo, siendo que para el caso de la discretización de Euler esta “colina” es menos pronunciada que en el caso de la discretización de Tustin donde, incluso, le sigue un “valle” como respuesta.

El resultado de la simulación en el experimento 4, considerando las condiciones $\omega_c = 100$, $k = 3$ y $n_{max} = 125$, es:

Similar al caso precedente, con un valor k más alto, la “colina” se exagera más, incluso haciendo que el patrón sinusoidal irregular se haga presente en la simulación con la discretización de Tustin.

Ante esto, podemos ir notando que la relación entre el valor de k y las frecuencias que el filtro mitiga se mantiene en ambos métodos de discretización, siendo esta relación directamente proporcional.

El resultado de la simulación en el experimento 5, considerando las condiciones $\omega_c = 100$, $k = 2,5$ y $n_{max} = 50$, es:

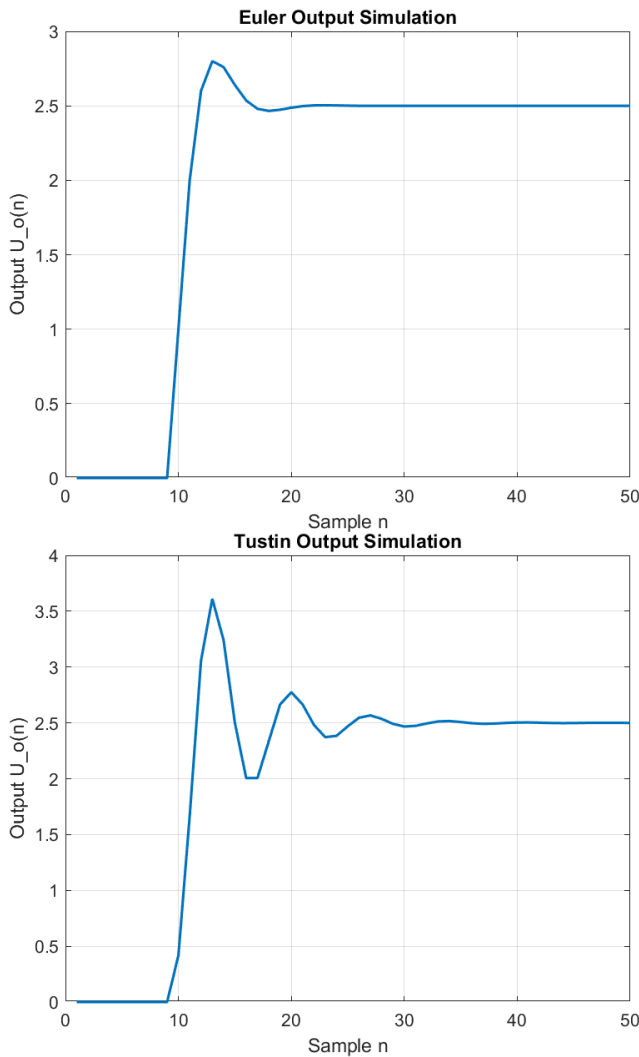


Figura 6: Simulación con experimento 5.

En este último experimento, con un valor medio tal que $2 < k < 3$, podemos notar cómo el comportamiento sinoidal se reduce, casi mitigándose en la simulación con discretización Euler y una notoria reducción en la tendencia sinoidal negativa.

V. CONCLUSIONES

Con los resultados obtenidos anteriormente, desde los primeros experimento podemos notar cómo los métodos de Euler y Tustin parecen simular las condiciones base de manera similar; mostrando que ambos métodos reconstruyen las mismas condiciones.

Así mismo, parecen responder de forma parecida ante la variación de la frecuencia de corte ω_c con ligeras variaciones en la “suavidad” con la que cambian su curvatura.

Sin embargo, ante variaciones pequeñas al valor k , los métodos muestran comportamientos distintos e, incluso, exagerados entre sí.

Recordando que el método de Euler no contempla valores imaginarios en contraste con el método de Tustin, entonces podemos notar cómo Tustin funciona mejor para reconstruir señales de respuesta que tienen un comportamiento periódico y responder mejor ante las frecuencias que la integran.

REFERENCIAS

- [1] Universidad de las Américas Puebla, *Capítulo II. Teoría de Filtros*. Universidad de las Américas Puebla (UDLAP). [En línea]. Disponible en: https://catarina.udlap.mx/u_dl_a/tales/documentos/lem/quiroz_c_g/capitulo2.pdf. Último acceso: 22 de abril de 2025.
- [2] S. Haykin and B. Van Veen, *Signals and Systems*, 2nd ed. Hoboken, NJ, USA: Wiley, 2005. Última consulta : 22 de abril de 2025.
- [3] A. S. Sedra and K. C. Smith, *Microelectronic Circuits*, 7th ed. New York, NY, USA: Oxford Univ. Press, 2016. Última consulta : 22 de abril de 2025.
- [4] B. C. Kuo and F. Golnaraghi, *Automatic Control Systems*, 8th ed. Hoboken, NJ, USA: Wiley, 2003.
- [5] J. G. Proakis and D. K. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2007. [En línea]. Disponible en: <https://mrce.in/ebooks/Digital%20Signal%20Processing%204th%20Ed.pdf>. Último acceso: 22 de abril de 2025.
- [6] G. F. Franklin, J. D. Powell, and A. Emami-Naeini, **Feedback Control of Dynamic Systems**, 7th ed. Boston, MA, USA: Pearson, 2015. Última consulta : 22 de abril de 2025.
- [7] Departamento de Mecánica, *Función de Transferencia*, FACULTAD DE CIENCIAS EXACTAS Y TECNOLOGIA, 2015. [En línea]. Disponible en: https://catedras.facet.unt.edu.ar/sistemasdecontrol/wp-content/uploads/sites/101/2015/12/CL03_Funci%u00f3n_Transferencia_Modelado_y_an%u00e1lisis_de_sistemas_2015.pdf. Último acceso: 22 de abril de 2025.
- [8] R. Ramos Lara, *Tema 6. Sistemas Digitales de Control en Tiempo Discreto*, 2007. [En línea]. Disponible en: <https://upcommons.upc.edu/bitstream/handle/2117/6123/TEMA6.pdf>. Último acceso: 22 de abril de 2025.

ANEXO A

Ecuación discreta de simulación de filtro pasa-bajas con método de Euler

$$U_o(z) = \frac{T^2 k U_i[z-0] \omega_c^2 - T k U_o[z-1] \omega_c + 3 T U_o[z-1] \omega_c + 2 U_o[z-1] - U_o[z-2]}{T^2 \omega_c^2 - T k \omega_c + 3 T \omega_c + 1}$$

Ecuación discreta de simulación de filtro pasa-bajas con método de Tustin

$$U_o(z) = - \frac{T^2 \omega_c^2 (2 U_o[z-1] + U_o[z-2] - k U_i[z] - 2 k U_i[z-1] - k U_i[z-2])}{T^2 \omega_c^2 - 2 T k \omega_c + 6 T \omega_c + 4} \\ + \frac{T \omega_c (2 k U_o[z-2] - 6 U_o[z-2]) - 8 U_o[z-1] + 4 U_o[z-2]}{T^2 \omega_c^2 - 2 T k \omega_c + 6 T \omega_c + 4}$$

ANEXO B. CÓDIGO DE SIMULACIÓN DE FILTRO PASA BAJAS CON DISCRETIZACIÓN EULER

```
1      % Constantes
2      T = 0.01;                % Periodo de muestreo
3      omega_c = 100;           % Frecuencia de corte
4      k = 1;                   % Valor arbitrario de k
5      n_max = 100;             % Numero de muestras
6
7      % Senal de entrada
8      U_i = zeros(1, n_max);
9      U_i(10:end) = 1;         % Paso inicial de entrada comenzando en n=10
10
11     % Senal de salida
12     U_o = zeros(1, n_max);
13
14     % Pre computacion de denominador (constante)
15     denominator = T^2 * omega_c^2 - T * k * omega_c + 3 * T * omega_c + 1;
16
17     % Simulacion iterativa para el sistema
18     for n = 3:n_max
19         numerator = T^2 * k * U_i(n) * omega_c^2 ...
20             - T * k * U_o(n-1) * omega_c ...
21             + 3 * T * U_o(n-1) * omega_c ...
22             + 2 * U_o(n-1) ...
23             - U_o(n-2);
24
25         U_o(n) = numerator / denominator;
26     end
27
28     % Grafica de salida (respuesta en el dominio del tiempo)
29     figure;
30     plot(U_o, 'LineWidth', 1.5)
31     xlabel('Sample n')
32     ylabel('Output U_o(n)')
33     title('Euler Output Simulation')
34     grid on
```

ANEXO C. CÓDIGO DE SIMULACIÓN DE FILTRO PASA BAJAS CON DISCRETIZACIÓN TUSTIN

```

1      % Constantes
2      T = 0.01;                % Periodo de muestreo
3      omega_c = 100;           % Frecuencia de corte
4      k = 1;                   % Valor arbitrario de k
5      n_max = 100;             % Numero de muestras
6
7      % Senal de entrada
8      U_i = zeros(1, n_max);
9      U_i(10:end) = 1;         % Paso inicial de entrada comenzando en k=10
10
11     % Senal de salida
12     U_o = zeros(1, n_max);
13
14     % Pre computacion de denominador (constante)
15     denominator = - T^2 * omega_c^2 + 2 * T * k * omega_c - 6 * T * omega_c
16         - 4;
17
18     % Simulacion iterativa para el sistema
19     for n = 3:n_max
20         numerator = 2 * T^2 * U_o(n-1) * omega_c^2 ...
21             + T^2 * U_o(n-2) * omega_c^2 ...
22             - T^2 * k * U_i(n) * omega_c^2 ...
23             - 2 * T^2 * k * U_i(n-1) * omega_c^2 ...
24             - T^2 * k * U_i(n-2) * omega_c^2 ...
25             + 2 * T * k * U_o(n-2) * omega_c ...
26             - 6 * T * U_o(n-2) * omega_c ...
27             - 8 * U_o(n-1) ...
28             + 4 * U_o(n-2);
29
30         U_o(n) = numerator / denominator;
31     end
32
33     % Grafica de salida (respuesta en el dominio del tiempo)
34     figure;
35     plot(U_o, 'LineWidth', 1.5)
36     xlabel('Sample n')
37     ylabel('Output U_o(n)')
38     title('Tustin Output Simulation')
39     grid on

```